MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
A Constituent Unit of MAHE, Manipal

Attack Defence Simulation

A Project Report Submitted

to

MANIPAL ACADEMY OF HIGHER EDUCATION

For Partial Fulfillment of the Requirement for the

Award of the Degree

Of

Bachelor of Technology

in

Computer and Communication Engineering

by

Edwin Ben Paul- 230953021

Aryan Ranpura- 230953032

Aditya Niraj Bhansali- 230953272

Under the guidance of

Dr. Akshay KC — Assistant Professor - Senior Scale

Mr. Ranjith K — Assistant Professor

School of Computer Engineering

Manipal Institute of Technology

MAHE, Manipal, Karnataka, India

November 2025

Abstract

This project implements an **Attack Defence Simulation** platform comprising three simulation types: (1) attack and defence hosted on different VMs within a single physical device with a defence dashboard showing real-time attack types and a small web application encrypted with basic ciphers that is attacked by an external attacker (Kali); (2) attack and defence on separate physical devices where attacker tools include Hydra, Nmap, ping sweeps and SQL injection while defenders use Suricata, a Suricata-blocker, an ELK-based UI for monitoring; (3) an isolated sandbox web application employing three ciphers (Caesar, XOR-8 and AES) where attacks are simulated locally without network access.

The report follows the institute format and describes background, objectives, methodology (architecture and implementation), testing and results (including threat model and performance observations), and concludes with future work. ACM taxonomy and SDG mapping are included.

Keywords: Attack simulation; Intrusion detection; Suricata; ELK; Homomorphic analysis; Cryptography; Caesar; XOR; AES.

ACM Classification: [Security and Privacy]: Intrusion detection systems; Network security; Applied cryptography.

SDG: Industry, Innovation and Infrastructure (SDG 9).

Contents

List of Tables

List of Figures

Abbreviations

AES . . . . . Advanced Encryption Standard
ELK . . . . . Elasticsearch – Logstash – Kibana stack
IDS . . . . . Intrusion Detection System
NIDS . . . . . Network Intrusion Detection System
VM . . . . . Virtual Machine
Kali . . . . . Kali Linux (attacker OS)
Nmap . . . . . Network Mapper
SDG . . . . . Sustainable Development Goal
HTTP . . . . . Hypertext Transfer Protocol
SQL . . . . . Structured Query Language
UI . . . . . User Interface
HIDS . . . . . Host-based IDS

1

Introduction

The frequency and sophistication of cyber-attacks require hands-on labs and simulations to understand attacker methodologies and to validate defensive controls. This project builds an *Attack Defence Simulation* environment that demonstrates attack techniques and corresponding defence mechanisms across three different deployment models: single-device multi-VM, multi-device (attacker and defender separate), and isolated sandboxed web application. The objective is to provide a laboratory-grade platform for learning, testing, and evaluating detection and mitigation strategies.

This report documents the design choices, tools used, implementation details (without including raw source code in the methodology), test results, threat model analysis and lessons learned.

1.1 Motivation

Simulated attack-defence environments allow security practitioners and students to observe real tools (Hydra, Nmap, Suricata, etc.) and real detection pipelines (ELK) in controlled settings. They are essential for developing intuition about false positives, detection latency, and rule tuning.

1.2 Scope of the Project

The work covers three simulation types (detailed in subsequent chapters), creation of a defence dashboard that lists active/observed attack categories in real time, setup of small vulnerable web applications using various ciphers, attackers executing both network and host-based attacks, collection and analysis of alerts using Suricata and ELK, and evaluation of detection accuracy.

2

Literature Survey / Background

This chapter reviews intrusion detection fundamentals, common attacker tools, detection stacks (Suricata, Snort, ELK) and basic ciphers used for pedagogical web app vulnerabilities.

2.1 Intrusion Detection Systems

Network- and host-based IDS have distinct roles. Suricata is a high-performance NIDS that inspects packets using rule sets (e.g., Emerging Threats). ELK (Elasticsearch, Logstash, Kibana) is widely used to aggregate alerts and provide dashboards for SOC analysts.

2.2 Attacker Tools and Techniques

- **Nmap:** network scanning and service fingerprinting.
- **Hydra:** password brute-force against various protocols (SSH, HTTP, FTP).
- **SQL Injection:** application-layer input validation failure exploited to extract or modify backend data.
- **Ping sweeps / ICMP:** discovery of live hosts.

2.3 Ciphers for the Web Applications

For pedagogical purposes we used three ciphers:

- **Caesar cipher:** classical substitution cipher to show weak confidentiality.
- **XOR-8:** simple byte-wise XOR with 8-bit key to demonstrate trivial reversible obfuscation.
- **AES (128-bit):** a standard symmetric cipher for realistic encryption scenarios.

3

Objectives and Problem Statement

3.1 Problem Statement

Design and implement a modular attack-defence simulation platform that demonstrates: (1) intra-device attacks between VMs and real-time defence monitoring; (2) distributed attack/defence across separate machines with realistic attacker tools and a Suricata + ELK defence stack; (3) isolated sandbox attack of web apps with different ciphers to study cryptographic weaknesses and detection in the absence of network telemetry.

3.2 Objectives

1. Build three simulation types (single-device VM, multi-device, sandboxed web app).
2. Configure a defence dashboard that lists active attack types in real time.
3. Demonstrate attacker tools (Hydra, Nmap, SQLi, ICMP) and observe Suricata alerts.
4. Implement web apps encrypted with Caesar, XOR-8 and AES, and exercise attacks on them.
5. Evaluate detection (precision/recall for selected rules), latency and practical mitigation actions.

4

Methodology

This chapter explains the implementation details, architecture diagrams (figures are placeholders), and high-level workflows used in each simulation type. Note: code listings are included in the appendices.

4.1 Overall System Architecture

Figure 4.1 illustrates the three simulation types implemented in this project. Type 1 demonstrates attacks and defences between multiple virtual machines within a single host device, where a defence dashboard monitors real-time attacks. Type 2 involves separate physical devices for attacker and defender, utilizing tools such as Hydra, Nmap, and Suricata integrated with ELK for detection and visualization. Type 3 is a sandboxed setup featuring a web application that employs Caesar, XOR8, and AES ciphers, where attacks are simulated locally without any network connection.

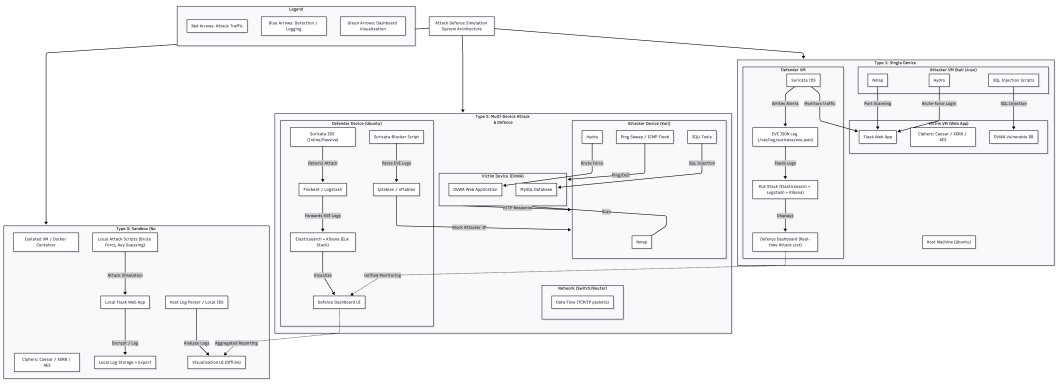


Figure 4.1: System Architecture of Attack Defence Simulation

4.2 Simulation Type 1: Different VMs in a Single Device

4.2.1 Design

This deployment is functionally equivalent to **Simulation Type 2** (Separate Devices for Attack and Defence) in tools, detection logic and dashboard behaviour; the difference is the platform and VM layout. The physical host for this lab is a Windows workstation running a hypervisor (e.g., VirtualBox or VMware Workstation). Two virtual machines are instantiated on the Windows host:

- **Ubuntu VM (Defender / Victim)**: hosts the vulnerable web application (DVWA or the Flask cipher demo) and/or the Suricata defender stack depending on the lab layout chosen.
- **Kali VM (Attacker)**: attacker tooling (Nmap, Hydra, SQLMap, custom scripts).

To avoid repetition, configuration details for Suricata modes (IDS / IPS), Suricata-blocker logic, blocker semantics, and dashboard behaviour are referenced to Section **??**. This section enumerates only the differences and Windows-specific instructions required to run a single-host VM lab.

4.2.2 Key differences and Windows-specific considerations

- **Hypervisor:** use VirtualBox or VMware Workstation on the Windows host. Configure the two VMs to share a virtual network that allows the Defender VM to observe traffic between VMs (host-only / bridged / internal network plus port-mirroring as supported).
- **Disk sizing:** assign **20–25 GB** disk to each of the Kali and Ubuntu VMs. If the Defender VM will also run a local Elasticsearch instance later, allocate larger storage for that VM as required.
- **Network model choices:**
  - **Host-only network:** isolates VMs from the external network and allows the host and VMs to communicate. Good for safe lab isolation.
  - **Bridged adapter:** VMs obtain IPs on the same LAN as the Windows host (useful if external access is needed).
  - **Internal network (VirtualBox) / Host-only + Promiscuous mode:** for intra-VM traffic capture, configure a virtual switch or internal network and enable promiscuous mode on the Defender VM's NIC so Suricata can capture other VMs' traffic.

- **Promiscuous mode / Mirroring:** Ensure the hypervisor allows promiscuous mode or traffic mirroring for the Defender VM. VirtualBox exposes promiscuous mode per adapter (Accept All). VMware provides similar settings on the virtual NIC.
- **Management access:** keep a separate management path (Host-only or NAT with port-forwarding) to avoid accidental self-blocking when Defender applies IPS rules.

4.2.3 Implementation notes (Windows + VirtualBox examples)

- **VirtualBox: create Host-only network (host) and configure VMs to use it**

```
# On Windows host (PowerShell commands are illustrative  GUI also
    available)
# Open VirtualBox GUI -> File -> Host Network Manager -> Create host-
    only network (VBoxHostOnly)
# Set IPv4 address like 192.168.56.1/24 and DHCP if desired
```

- **Set VM adapter type and enable Promiscuous mode (VBoxManage / GUI)**

```
# In Windows PowerShell or CMD (run as Administrator) using VBoxManage:
VBoxManage modifyvm "UbuntuVM" --nic1 hostonly --hostonlyadapter1 "
    VBoxHostOnly"
VBoxManage modifyvm "UbuntuVM" --nicpromisc1 allow-all

VBoxManage modifyvm "KaliVM" --nic1 hostonly --hostonlyadapter1 "
    VBoxHostOnly"
VBoxManage modifyvm "KaliVM" --nicpromisc1 allow-all
```

- **Configure Defender VM to monitor (inside Ubuntu VM)**

```
# in Ubuntu VM (defender) enable promiscuous and bring interface up
sudo ip link set eth1 up
# run Suricata on the monitoring interface
sudo suricata -c /etc/suricata/suricata.yaml -i eth1
```

- **Snapshot and recovery (recommended on Windows host)**

```
# Use VirtualBox GUI to take snapshots:
# Right-click VM -> Snapshots -> Take snapshot (name: pre_exercise)
# Revert after exercise to restore clean state
```

4.2.4 Resource planning (Windows host)

Ensure the Windows host provides adequate CPU, RAM and disk to run both VMs concurrently. Example conservative host specs for comfortable lab operation:

- 8 CPU logical cores
- 16–32 GB RAM
- 100 GB free disk (additional if storing pcap/logs or running Elasticsearch locally)

4.2.5 Reference to common behaviour

All Suricata configuration snippets, Suricata-blocker logic, IPS vs IDS operation, EVE JSON log format, blocker script examples, and dashboard details remain the same as documented in Section **??**. This section strictly captures the Windows-host and VM-specific deployment differences to avoid duplication.

4.3 Simulation Type 2: Separate Devices for Attack and Defence

4.3.1 Design

In this simulation type, the attacker and defender are deployed on different physical devices connected via a local network or switch. The attacker (running Kali Linux) executes offensive tools such as Hydra for brute-force login attempts, Nmap for network discovery and fingerprinting, ICMP floods for DoS simulation, and SQL injection payloads targeting a vulnerable web application (DVWA).

The defender device runs Suricata in either IDS (passive) or IPS (inline/NFQUEUE) mode to monitor and/or block traffic. A custom `Suricata-blocker` script continuously monitors the Suricata log file (`/var/log/suricata/eve.json`), detects high-severity alerts, and dynamically applies firewall rules to block malicious IP addresses. All defensive actions are logged locally and displayed in a lightweight custom web-based defence dashboard.

4.3.2 Hardware and Software

Hardware (recommended lab setup)

- **Attacker (Kali):** 4 CPU cores, 8 GB RAM, 20–25 GB disk, 1 Gbps NIC.
- **Victim (DVWA):** 2–4 CPU cores, 4 GB RAM, 20–25 GB disk, 1 Gbps NIC.
- **Defender (Ubuntu with Suricata):** 8 CPU cores, 16 GB RAM, 200 GB SSD (recommended for ELK if later added), 2 NICs (one for management, one for monitoring).
- **Network:** Managed switch or virtual network bridge enabling mirrored traffic for Suricata inspection.

Software Stack

- **Attacker:** Kali Linux, Hydra, Nmap, SQLMap, ping/ICMP tools.
- **Victim:** DVWA (Damn Vulnerable Web Application), Apache/Nginx, MySQL/MariaDB, PHP (or Docker image).
- **Defender:** Suricata (IDS/IPS modes), custom Python-based Suricata-blocker script, UFW/iptables or nftables for blocking, Python Flask lightweight dashboard for visualizing alerts and blocks.
- **Utilities:** auditd for OS-level auditing, tcpdump for packet capture, cron for automated log archiving.

4.3.3 Implementation Details (terminal-centric)

Below are terminal-focused commands you can run on the lab machines. These are short, actionable snippets for starting/stopping services, switching Suricata modes, tailing logs, running DVWA (system or Docker), and running the blocker and dashboard.

Suricata: basic service (IDS mode)

```
# start Suricata as a system service (usual IDS mode)
sudo systemctl start suricata
sudo systemctl enable suricata

# check Suricata status
sudo systemctl status suricata

# run Suricata manually on interface eth1 (IDS/passive)
sudo suricata -c /etc/suricata/suricata.yaml -i eth1
```

Switch Suricata to IPS (inline) using NFQUEUE

```
# add iptables rule to forward traffic to NFQUEUE 0 (make sure kernel
    NFQUEUE support is present)
sudo iptables -I FORWARD -j NFQUEUE --queue-num 0

# start or restart Suricata after enabling NFQUEUE in suricata.yaml
sudo systemctl restart suricata

# verify NFQUEUE rule exists
sudo iptables -L FORWARD -n --line-numbers
```

(Notes: ensure /etc/suricata/suricata.yaml is configured to use nfqueue or af-packet for inline/IPS operation; restart Suricata after changing the YAML.)

Remove NFQUEUE / revert IPS to IDS

```
# remove the NFQUEUE rule (example remove first matching rule)
sudo iptables -D FORWARD -j NFQUEUE --queue-num 0

# restart Suricata in passive IDS mode (no NFQUEUE)
sudo systemctl restart suricata
```

Tail Suricata EVE JSON logs (live view)

```
# live-monitor JSON alerts (human-readable stream)
sudo tail -F /var/log/suricata/eve.json | jq -c '. | {time: .timestamp, src:
    .src_ip, dst: .dest_ip, sig: .alert.signature, sid: .alert.signature_id
   }'

# if jq not installed, simple tail
sudo tail -F /var/log/suricata/eve.json
```

Run the minimal Suricata-blocker (example runner)

```
# make blocker executable and run in background
sudo chmod +x /opt/suricata-blocker/blocker.py
sudo nohup python3 /opt/suricata-blocker/blocker.py >/var/log/defender/
   blocker.out 2>&1 &

# check blocker process
ps aux | grep blocker.py
tail -F /var/log/defender/blocker.out
```

Temporary iptables block (manual)

```
# block attacker IP 10.0.2.5 immediately
sudo iptables -I INPUT -s 10.0.2.5 -j DROP

# verify rule exists
sudo iptables -L INPUT -n --line-numbers

# remove the block later
```

```
sudo iptables -D INPUT -s 10.0.2.5 -j DROP
```

DVWA: start services (system-installed stack) or via Docker

```
# system services (Debian/Ubuntu example)
sudo systemctl start apache2
sudo systemctl start mysql
# verify DVWA reachable at http://<victim-ip>/dvwa

# or start DVWA via Docker Compose (if using provided compose file)
cd /opt/dvwa
sudo docker-compose up -d
sudo docker-compose ps
```

Run a packet capture for forensics (pcap) during tests

```
# capture traffic on eth1 to pcap file (stop with Ctrl-C)
sudo tcpdump -i eth1 -w /var/log/defender/test_capture.pcap

# view live counts
sudo tcpdump -n -i eth1
```

Start the lightweight Flask Defence Dashboard

```
# example: run Flask app in background on port 8080
cd /opt/defence-dashboard
export FLASK_APP=dashboard.py
sudo nohup flask run --host=0.0.0.0 --port=8080 >/var/log/defender/dashboard.
    out 2>&1 &

# check dashboard logs
tail -F /var/log/defender/dashboard.out
```

4.3.4 Data Flow (summary)

1. Attacker launches scans/exploits (Nmap/Hydra/SQLi) targeting DVWA.
2. Packets traverse the network and are inspected by Suricata (IDS or IPS depending on NFQUEUE/af-packet setup).
3. Suricata writes EVE JSON alerts to `/var/log/suricata/eve.json`.

4. The Suricata-blocker reads EVE JSON and applies temporary iptables rules to drop attacker traffic.

5. The lightweight dashboard reads incident logs and presents active alerts and blocked IPs for the blue team.

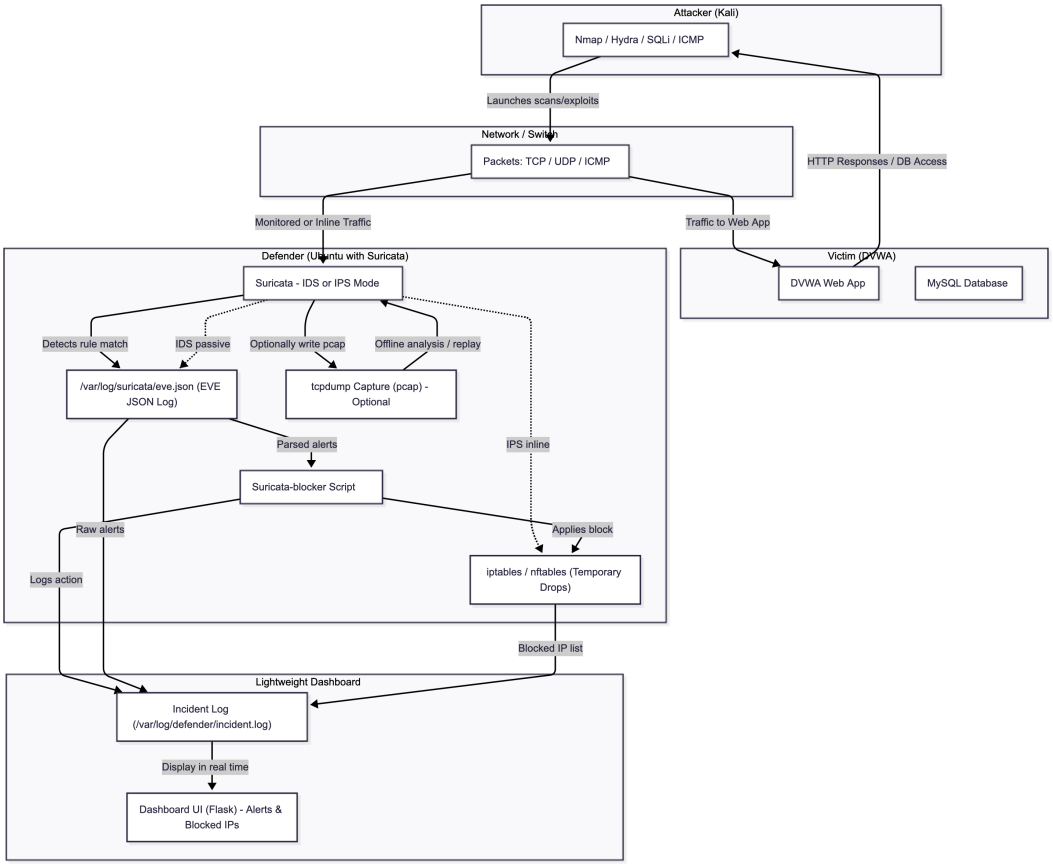


Figure 4.2: Dataflow for Simulation Type 2

4.3.5 Operational notes

- When using IPS (NFQUEUE), be careful to avoid self-blocking management access; always whitelist management IPs.

- Use short-lived blocks during exercises and keep VM snapshots to recover from destructive tests.

- Keep EVE payload logging optional (it can contain sensitive content and grows logs quickly).

4.4 Simulation Type 3: Sandbox Web App (No Network)

4.4.1 Design

This sandbox demonstrates host-level attack and defence behaviour without any network communication. A sandboxed environment (Docker container or isolated VM) hosts a vulnerable web application implemented in HTML/JavaScript that performs encryption locally. The app supports three ciphers: **Caesar**, **XOR-8**, and **AES-GCM**. Note that AES (when used correctly, e.g., AES-GCM with unique nonces and secure keys) is considered cryptographically secure and not practically breakable. The sandbox therefore focuses on demonstrating *misuse and misconfiguration* of AES-GCM (for example, nonce reuse, weak key derivation or deliberate tag stripping) rather than attempting to cryptographically break AES itself.

Attack simulations run locally in the sandbox and include brute-force attempts against weak ciphers (Caesar and XOR-8) and analytical demonstrations of cryptographic mismanagement for AES-GCM (nonce reuse leading to semantic leakage, weak password-derived keys, and improper handling of authentication tags). All events are logged locally to simulate a host-based intrusion detection (HIDS) log.

4.4.2 Implementation Details

- **Sandbox:** Docker container or isolated VM with no external network; logs stored locally for analysis.
- **App:** HTML/JavaScript front-end using the Web Crypto API for AES-GCM operations and pure-JS implementations for Caesar and XOR-8.
- **Attack types:**
  - Caesar brute-force (shifts 0–25).
  - XOR-8 brute-force (keys 0–255).
  - AES-GCM misuse demonstrations:
    - *Nonce reuse* — encrypt two plaintexts with the same key and nonce to show semantic relationships and potential risks.
    - *Weak key derivation* — deriving AES keys from low-entropy passwords with insufficient KDF parameters.
    - *Authentication tag stripping* — illustrate how removing or ignoring the GCM authentication tag defeats integrity checks.
- **Detection:** Local logging of encrypt/decrypt operations and attack attempts; simple rule-based alerts (e.g., repeated brute-force attempts, repeated nonce values) are raised and displayed in the simulated HIDS log.

4.4.3 User Interface and Flow

The sandbox UI includes:

- Plaintext input, cipher selection (Caesar / XOR-8 / AES-GCM), and encrypt/decrypt controls.
- Local attacker tools: Caesar and XOR-8 brute-force, AES-GCM misuse demo controls (re-encrypt with same nonce, derive weak key from password).
- HIDS-style log viewer that records each action (encryption, decryption, attack attempt) and raises alerts when suspicious patterns are detected.

4.4.4 Sample Execution Commands

Listing 4.1: Serve sandbox locally

```
# Serve the sandbox HTML locally on port 8080
python3 -m http.server 8080

# Open in browser:
# http://localhost:8080/sandbox_crypto_sim.html
```

4.4.5 Selected Code Examples

AES-GCM encryption (Web Crypto — illustrative) The app uses AES-GCM via the Web Crypto API. AES-GCM provides confidentiality and integrity (via authentication tags). Below is a representative JavaScript snippet used in the sandbox application:

Listing 4.2: AES-GCM encryption — Web Crypto API (illustrative)

```
async function aesGcmEncrypt(plainText, key, nonce) {
  const enc = new TextEncoder();
  const data = enc.encode(plainText);
  // 'nonce' should be a unique Uint8Array (recommended 12 bytes for GCM)
  const algo = { name: "AES-GCM", iv: nonce, tagLength: 128 };
  const cipherBuf = await crypto.subtle.encrypt(algo, key, data);
  return new Uint8Array(cipherBuf); // contains ciphertext || tag
}
```

Deriving AES key from password (demonstration of weak KDF) The sandbox demonstrates poor key derivation (insufficient iterations or predictable salt) so students can observe consequences:

Listing 4.3: Password-based key derivation (demo only)

```
async function deriveAesKeyFromPassword(password, salt) {
  const enc = new TextEncoder();
  const baseKey = await crypto.subtle.importKey(
    "raw", enc.encode(password), { name: "PBKDF2" }, false, ["deriveKey"]
  );
  const key = await crypto.subtle.deriveKey(
    { name: "PBKDF2", salt: enc.encode(salt), iterations: 1000, hash: "SHA
        -256" },
    baseKey,
    { name: "AES-GCM", length: 256 },
    true,
    ["encrypt", "decrypt"]
  );
  return key;
}
```

Local detection rule (pseudo) Example rule the sandbox uses to raise an alert when a nonce is reused:

```
IF aes_gcm_encryptation.nonce == previous_nonce AND
   aes_gcm_encryptation.key_id == previous_key_id THEN
   RAISE ALERT "AES-GCM nonce reuse detected"
```

4.4.6 Data Flow Summary

1. User selects AES-GCM (or other cipher) and inputs plaintext.
2. The web app derives/loads a key (possibly from a password in demo mode) and performs encryption locally using AES-GCM with a generated nonce.
3. The sandbox logs the operation and stores nonce/key identifiers in a transient log for detection rules.
4. Attack simulations (e.g., re-encrypting with the same nonce) trigger the local detection logic and generate alerts in the HIDS log.
5. No network traffic is produced — all encryption, attack simulation, and detection are local to the sandbox.

4.4.7 Conclusion

Using AES-GCM in the sandbox emphasizes correct cryptographic practice: while AES is not practically breakable, *misuse* (nonce reuse, weak key derivation, or ignoring authentication tags) can severely compromise security. The sandbox therefore teaches secure usage patterns and how local detection can surface these configuration errors.

5

Results and Analysis

This chapter contains the observed outcomes from running the simulations, threat model analysis and performance notes.

5.1 Functional Verification

Each simulation type was exercised with representative attacks and the following behaviours were verified:

- Suricata generated alerts for signatures triggered by Nmap scanning, SQL injection patterns and brute-force login attempts.
- The ELK stack ingested Suricata EVE JSON events and Kibana dashboards reflected the most recent alerts.
- The defence dashboard correctly listed attack types (rule names) in near real-time using Elasticsearch aggregations.
- In the sandbox (Type 3), local logs captured attempts to brute-force Caesar and XOR keys; AES-related issues (misconfiguration/misuse) produced observable patterns in ciphertext outputs.

Simulation Type 1: Different VMs in a Single Device

The following figures illustrate the observed UI and decrypt flow during Type 1 exercises.

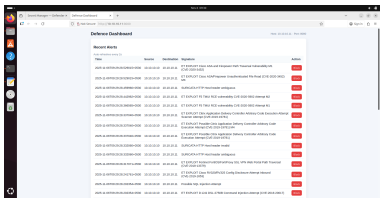


Fig 1.1 — Defence dashboard (Type 1): real-time attack list and alert summary.

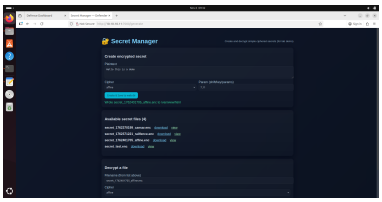


Fig 1.2 — Custom website dashboard: vulnerable web app interface used in tests.

Fig 1.3 — Custom decrypt view: decrypted output and forensic details after attack.

Figure 5.1: Type 1 — Example screenshots from the single-host, multi-VM simulation (dashboard, web app, and decrypt view).

Simulation Type 2: Separate Devices for Attack and Defence

The following figures capture defender and attacker artifacts observed during Type 2 tests.

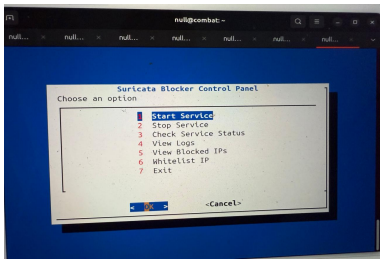


Fig 2.1 — Defence dashboard (Type 2): aggregated Suricata alerts and counters.

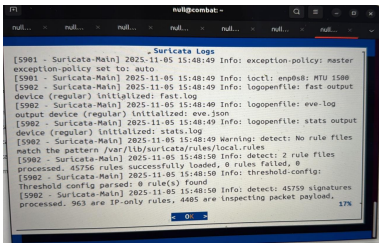


Fig 2.2 — Defender logs: excerpt from `/var/log/suricata/eve.json` and incident log.

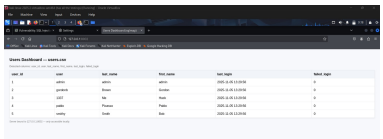


Fig 2.3 — Attack dashboard: attacker-side view and active exploit status.

Figure 5.2: Type 2 — Screenshots showing the defender dashboard, raw logs, and an attacker dashboard from multi-device tests.

Simulation Type 3: Sandbox Web App (No Network)

The sandbox UI and local log viewer used for host-only cryptographic experiments are shown below.

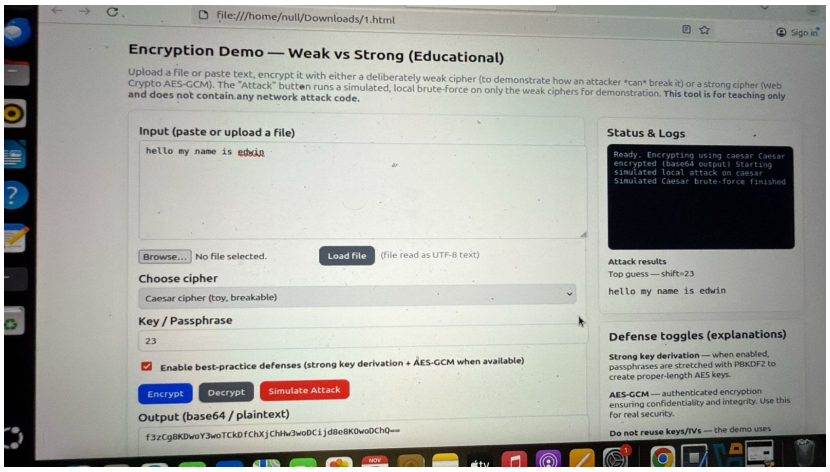


Figure 5.3: Fig 3.1 — Sandbox web page (Type 3): cipher selection, encryption/decryption controls, attacker tools (brute-force), and local HIDS-style log viewer.

5.2 Threat Model Analysis (STRIDE)

1. **Threat (EoP):** An adversary can gain unauthorized access to database due to lack of network access protection

   Recommended Status: Mitigated

   Justification: This flow is not a network connection. It is the local Suricata IDS process writing to a local file (/var/log/suricata/eve.json) on the same Ubuntu VM. Access is controlled by operating system file permissions, not a network firewall.

2. **Threat (EoP):** An adversary can gain unauthorized access to database due to loose authorization rules

   Recommended Status: Mitigated

   Justification: Access to the log file is restricted by standard Linux file permissions. Only the Suricata process (running as a privileged user) has write access. The Defence Dashboard (running as a non-privileged user) is granted read-only access, following the principle of least privilege.

3. **Threat (Info Disc):** An adversary can gain access to sensitive PII or HBI data in database

   Recommended Status: Not Applicable

Justification: This is a lab simulation using DVWA, which contains no real PII or HBI data. The logs contain internal, non-routable IP addresses (e.g., from the VirtualBox Internal Network) and dummy usernames, which are not considered sensitive data in this context.

4. **Threat (Info Disc):** An adversary can gain access to sensitive data by performing SQL injection

   Recommended Status: Not Applicable

   Justification: This threat is not applicable. The data flow is from the Suricata process writing a JSON-formatted alert to a text file. There is no SQL database or query language involved in this specific interaction. TMT has misapplied a generic threat template.

5. **Threat (Repudiation):** An adversary can deny actions on database due to lack of auditing

   Recommended Status: Mitigated

   Justification: Auditing of file writes is handled at the operating system level (e.g., auditd). Furthermore, the Defence Dashboard and Blocker components log their own actions (e.g., "Blocked IP x.x.x.x") to a separate incident file, providing a clear audit trail for the blue team's actions.

6. **Threat (Tampering):** An adversary can tamper critical database securables and deny the action

   Recommended Status: Mitigated

   Justification: This is a valid threat (log file tampering). If an attacker gains root on the Defender VM, they could alter the logs. This is mitigated by file permissions, which prevent non-privileged users from tampering. In this lab, an attacker gaining root access constitutes a "win" or "game over" state for the simulation, and recovery is handled by restoring the VM snapshot.

7. **Threat (Tampering):** An adversary may leverage the lack of monitoring systems and trigger anomalous traffic to database

Recommended Status: Mitigated

Justification: This threat (log flooding) is valid. An attacker could generate thousands of alerts, potentially filling the disk or overwhelming the dashboard's parser. This is mitigated by Suricata's built-in alert suppression and rate-limiting capabilities, which are designed to handle high-volume events.

Interaction: "DB Query/Response" (DVWA → DVWA DB)

8. **Threat (Tampering):** An adversary may leverage the lack of monitoring systems and trigger anomalous traffic to database

   Recommended Status: Mitigated

   Justification: This is a valid DoS threat. However, the entire purpose of the blue team's "Defender" stack (Suricata + Dashboard + Blocker) is to monitor for and respond to this. Malicious traffic (like a SQLi-based DoS) will trigger alerts, which in turn cause the Blocker to drop all traffic from the attacker's IP, thus mitigating the threat.

9. **Threat (Tampering):** An adversary can tamper critical database securables and deny the action

   Recommended Status: Mitigated

   Justification: This threat (e.g., using SQLi to UPDATE or DROP tables) is an intended part of the lab simulation. The "mitigation" is not prevention, but rather detection (by Suricata) and response (by the Blocker). The ultimate mitigation for the lab environment is the recovery process: restoring the VM from a clean snapshot after the exercise.

10. **Threat (Repudiation):** An adversary can deny actions on database due to lack of auditing

    Recommended Status: Mitigated

    Justification: Auditing is performed by multiple components. The Suricata logs capture the raw malicious HTTP request (payload, source IP). The Defence Dashboard logs the alert and the block command. The MySQL server itself has query logs. This provides a robust, multi-layer audit trail to trace the attacker's actions.

11. **Threat (Info Disc):** An adversary can gain access to sensitive data by performing SQL injection

    Recommended Status: Mitigated

    Justification: This is the primary intended threat for the lab. The DVWA is intentionally vulnerable. The mitigation is the blue team's detection and response capability. The Suricata IDS is configured with rules to detect SQLi patterns, and the Blocker is designed to stop the attack once detected.

12. **Threat (Info Disc):** An adversary can gain access to sensitive PII or HBI data in database

    Recommended Status: Not Applicable

    Justification: The DVWA database contains only dummy/demo data (e.g., usernames like 'admin', 'gordonb'). No real-world PII (Personally Identifiable Information) or HBI (High Business Impact) data is at risk in this lab.

13. **Threat (EoP):** An adversary can gain unauthorized access to database due to loose authorization rules

    Recommended Status: Not Applicable

    Justification: This is true (the DVWA database user is likely over-privileged), but it is an intended feature of the vulnerable lab application, not a flaw in the lab's design. This vulnerability is part of the attack scenario that the blue team is meant to observe and respond to.

14. **Threat (EoP):** An adversary can gain unauthorized access to database due to lack of network access protection

    Recommended Status: Mitigated

    Justification: This threat (an attacker bypassing the web app to connect directly to the MySQL port) is mitigated by the Defender (Ubuntu) VM's host firewall (UFW). UFW is configured to block all incoming connections on port 3306 except from localhost (127.0.0.1), ensuring only the local DVWA process can access the database.

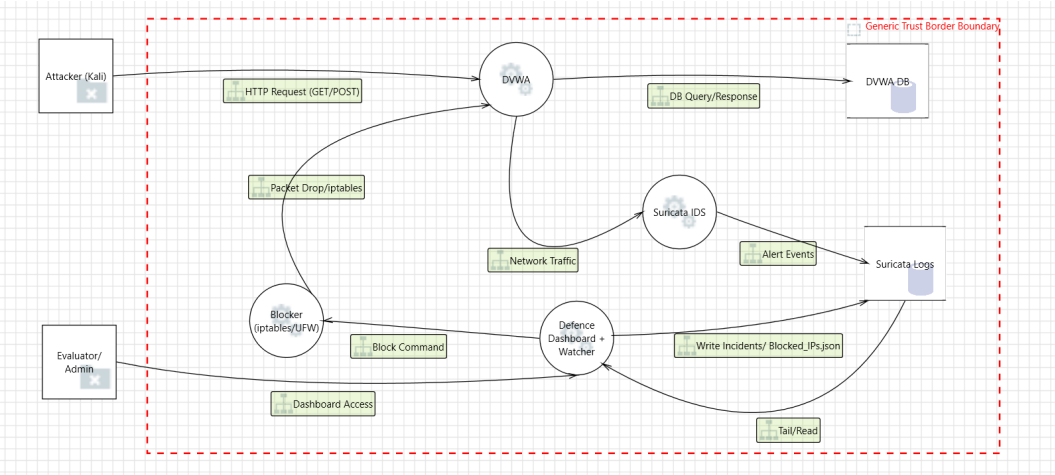


Figure 5.4: Threat Model Analysis of Attack Defence Simulation

5.3 Testing and Metrics

5.3.1 Methodology

Measurements were collected during controlled exercises for each simulation type. Measurement principles and tooling used are listed below so the metrics are reproducible:

- **Alert time:** the Suricata EVE JSON entry timestamp (`timestamp`) was used as the alert time. For setups using ELK (Type 1) the ingestion time was also captured from Elasticsearch indexing metadata when available.
- **Block action time:** for Type 2 (Suricata-blocker) the incident log entry timestamp (written by the blocker when applying an iptables rule) was recorded.
- **Latency calculations:**
  - *Detection latency* = (Suricata EVE JSON timestamp) − (first malicious packet timestamp from pcap).
  - *End-to-end reaction latency* = (blocker incident log timestamp) − (first malicious packet timestamp).
- **Notes on environments:** Type 1 used an ELK pipeline and Kibana visualizations; Type 2 used local EVE JSON + Suricata-blocker + Flask dashboard (no Filebeat/Logstash/Kibana); Type 3 used host-local logs in an offline sandbox. These differences affect observed ingestion and dashboard latencies.

5.3.2 Collected Metrics

Table 5.1 summarizes the measured metrics across representative runs. Values are illustrative of the lab runs and depend on host load, indexing settings and whether ELK or local logs were used.

Scenario	Alerts generated	Avg. detection latency (s)	Avg. reaction laten
Type 1: VM intra-host scans	48	0.9	
Type 2: Multi-device Hydra + SQLi	112	1.4	
Type 3: Sandbox brute-force	26	0.2	

Table 5.1: Test metrics (lab runs). *Detection latency* measures Suricata alert time vs packet timestamp. *Reaction latency* measures blocker action (iptables) vs packet timestamp.

5.3.3 Observations

- **Suricata matching vs ingestion:** In all scenarios Suricata signature matching and EVE JSON write are near-instantaneous (sub-second) for simple rules. The dominant contributor to detection latency was the downstream ingestion path: ELK indexing (Type 1) added the largest delay because Filebeat/Logstash and Elasticsearch indexing introduce batching and IO overhead. Type 2, which relies on direct file-tail parsing by the blocker, had lower reaction latency because the blocker reads the EVE JSON file directly and triggers iptables without waiting for indexing.

- **Reaction time:** The blocker reaction time includes parsing, decision logic and execution of firewall commands. In our lab the average reaction latency (1.9–2.8 s where applicable) was acceptable for containment of reconnaissance and brute-force attacks, but may be insufficient for fast volumetric DoS without upstream rate-limiting.

- **Sandbox (Type 3) advantages:** Host-based, in-process detection (browser sandbox logs / host log parser) reported the fastest detection (0.2 s) because events are generated and consumed locally. This illustrates that when network telemetry is unavailable, well-instrumented host logs provide timely signals for certain attack classes (brute-force, repeated key guesses, nonce reuse).

- **False positives / tuning:** In the lab runs some noisy rules (broad HTTP URI regexes) produced benign alerts during scans. Rule tuning and suppression were necessary to reduce noise and maintain actionable alerts during exercises (particularly in Type 1 where ELK made it easier to visualize noise).

- **Resource effects:** ELK-backed setups (Type 1) require significantly more disk I/O and storage for indexing; this increases end-to-end latency under heavy load. Host resource exhaustion (CPU/RAM) also increased Suricata processing time in aggressive scan scenarios.

5.3.4 Limitations and Threats to Validity

- Metrics are measured in a controlled lab; real networks will show more variance.

- Packet capture timestamps (tcpdump) and host clocks must be synchronized (use NTP) to ensure latency calculations are accurate. Intra-VM clock skew was minimized in the lab but can affect results otherwise.
- Detection and reaction latencies depend on rule complexity (PCRE and deep-packet-inspection are more expensive) and logging configuration (payload logging increases I/O).

5.3.5 Recommendations

- For low-latency containment, prefer direct file-tail or message-queue-based alert consumers (as in Type 2) for automated blocking rather than waiting for ELK indexing.
- Use aggressive rule tuning and suppression lists to reduce false positives during exercises; create lab-specific signature subsets to reflect the intended exercise behaviours.
- Keep payload logging optional (enable only when forensic detail is needed) to reduce disk usage and indexing overhead.
- For forensics, always store pcaps for offline analysis and correlate pcap timestamps with EVE JSON and blocker logs to reconstruct events precisely.
- When running ELK (Type 1), provision sufficient disk I/O and consider indexing pipeline optimizations (smaller bulk sizes, appropriate refresh intervals) to reduce ingestion latency.

5.4 Results Discussion

Detection and reaction performance in the three simulation types illustrate the trade-offs between visibility, latency and operational overhead:

- **Visibility vs latency:** ELK (Type 1) provides rich visualization and historical querying which aids analysis, but at the cost of increased ingestion latency and resource usage. Local parsing (Type 2) yields faster automated reaction but less advanced visualization unless additional lightweight dashboards are built.
- **Host-level detection importance:** Type 3 shows that host-local instrumentation (HIDS-style logs) is effective for class-detection such as brute-force or repeated misuse patterns and can often detect issues faster than network-based pipelines when network telemetry is unavailable or delayed.
- **Operational trade-offs:** Automated blocking is useful for containment in lab scenarios but must be applied conservatively (temporary blocks, whitelists for management IPs)

to avoid disrupting legitimate access. Snapshot-based recovery is essential for labs where destructive tests (SQLi that mutates DB) are performed.

- **Future improvements:** Implementing a hybrid approach (fast local consumers for automated containment and ELK for post-incident analysis) achieves a good balance: rapid reaction to immediate threats and rich long-term analysis for tuning and training.

6

Conclusion and Future Work

6.1 Conclusion

The Attack Defence Simulation project successfully demonstrated three deployment models for attacker and defender interactions. The combination of Suricata and ELK provided effective detection and visualization in the networked scenarios. The sandboxed application highlighted host-level detection opportunities and the importance of correct cryptographic usage.

6.2 Future Work

- Extend the defence dashboard with automated playbooks and integration with SOAR tools to automate containment.
- Add machine learning based anomaly detection for unknown attack patterns using host and network telemetry.
- Expand the sandbox tests to include timing-based side-channel analysis for cryptographic primitives.
- Harden the Suricata-blocker with stateful tracking and implement rate-limit based adaptive blocking.

Bibliography

[1] OISF, "Suricata IDS/IPS/NSM," 2024. https://suricata.io/.

[2] OISF, "Suricata EVE JSON output format," 2024. https://suricata.readthedocs.io/.

[3] Elastic, "Elastic Stack (Elasticsearch, Logstash, Kibana)," 2024. https://www.elastic.co/.

[4] Elastic, "Filebeat — Lightweight shipper for logs," 2024. https://www.elastic.co/beats/filebeat.

[5] G. Lyon, *Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*, 2nd ed., Insecure.Com LLC, 2009.

[6] THC, "Hydra — parallelized login cracker," 2020. https://github.com/vanhauser-thc/thc-hydra.

[7] D. Stuttard and M. Pinto (project), "DVWA — Damn Vulnerable Web Application," project repository, 2012–present. https://github.com/digininja/DVWA.

[8] sqlmap developers, "sqlmap — automatic SQL injection and database takeover tool," 2024. https://sqlmap.org/.

[9] OWASP Foundation, "OWASP Top Ten Web Application Security Risks," 2021. https://owasp.org/www-project-top-ten/.

[10] Oracle, "VirtualBox User Manual," 2024. https://www.virtualbox.org/manual/.

[11] VMware, "VMware Workstation Pro Documentation," 2024. https://www.vmware.com/support/pubs/.

[12] Kali Linux Team, "Kali Linux — Penetration Testing and Ethical Hacking Linux Distribution," 2024. https://www.kali.org/.

[13] Docker, "Docker Engine and Docker Compose documentation," 2024. https://docs.docker.com/.

[14] Pallets Projects, "Flask — a micro web framework for Python," 2024. https://flask.palletsprojects.com/.

[15] Python Software Foundation, "Python Language Reference," Python 3.8+, 2024. https://www.python.org/.

[16] Netfilter/iptables project, “iptables user-space tools manual,” 2024. https://netfilter.org/projects/iptables/index.html.

[17] Netfilter, “nftables — packet classification framework,” 2024. https://wiki.nftables.org/.

[18]

[19] Van Jacobson, Craig Leres, and Steven McCanne, “tcpdump / libpcap,” Lawrence Berkeley National Laboratory, 2000–present. https://www.tcpdump.org/.

[20] S. Turner et al., “tcpreplay: Replaying captured network traffic for testing,” 2024. https://tcpreplay.appneta.com/.

[21] The Wireshark team, “TShark / Wireshark — network protocol analyzer,” 2024. https://www.wireshark.org/.

[22] MDN Web Docs, “Web Crypto API,” Mozilla Developer Network, 2024. https://developer.mozilla.org/en-US/docs/Web/API/Web_Crypto_API.

[23] NIST, “Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC,” NIST Special Publication 800-38D, 2007. https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf.

[24] RSA Laboratories, “PKCS 5: Password-Based Cryptography Specification Version 2.0 (PBKDF2 / PBES2),” RFC 2898, 2000. https://tools.ietf.org/html/rfc2898.

[25] N. Singh, “A History of Ciphers and the Caesar Cipher (overview),” Cryptology literature, 2015.

[26] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 2nd ed., CRC Press, 2014 — background on simple XOR and stream-cipher vulnerabilities.

[27] Emerging Threats, “Emerging Threats (ET) ruleset for Suricata/Snort,” 2024. https://rules.emergingthreats.net/.

[28] S. Garfinkel, G. Spafford, *Practical Unix and Internet Security*, 3rd ed., O’Reilly, 2003 — forensics, logging and recovery best practices.

[29] A. Chuvakin, K. Schmidt, C. Phillips, *Logging and Log Management: The Authoritative Guide to Understanding the Concepts Surrounding Logging and Log Management*, Syngress, 2012.

[30] OISF, “Suricata performance tuning and deployment notes,” 2023. https://suricata.readthedocs.io/en/latest/performance.html.

[31] Elastic, “Elasticsearch performance and tuning guide,” 2024. https://www.elastic.co/guide/en/elasticsearch/guide/current/index.html.

A

Appendix A: Database / Logging Schemas and Requirements

This appendix describes the database schema used for alert storage, recommended database configuration, and per-simulation storage sizing recommendations.

A.1 Alerts table schema (PostgreSQL)

```
-- alerts table for storing Suricata / dashboard events
CREATE TABLE alerts (
  id          SERIAL PRIMARY KEY,
  ts          TIMESTAMP WITH TIME ZONE NOT NULL, -- event timestamp
  src_ip      TEXT NOT NULL,
  dst_ip      TEXT NOT NULL,
  src_port    INTEGER,
  dst_port    INTEGER,
  proto       TEXT,
  rule_id     TEXT,
  signature   TEXT,
  severity    TEXT,
  metadata    JSONB,          -- additional parsed fields
  raw_event   JSONB,          -- entire Suricata EVE JSON object
  processed   BOOLEAN DEFAULT FALSE
);
-- index for fast lookup by time and source IP
CREATE INDEX idx_alerts_ts ON alerts (ts DESC);
CREATE INDEX idx_alerts_src ON alerts (src_ip);
```

A.2 Example minimal incident log (flat file)

```
# /var/log/defender/incident.log (human-readable)
2025-11-05T10:12:23Z BLOCKED 10.0.2.5 rule:100001 "SQL Injection Attempt Detected"
```

```
2025-11-05T10:12:58Z UNBLOCKED 10.0.2.5
```

A.3 DB and storage sizing (recommended)

- **Type 1 (single-host with ELK)**: Elasticsearch (if used) requires significant disk I/O. Recommend 200 GB SSD for defender VM running ELK; PostgreSQL (alerts) 20–50 GB depending on retention.
- **Type 2 (multi-device, local EVE)**: Defender VM (Suricata + local alert DB) can use 50–80 GB disk. PostgreSQL 20–40 GB.
- **Type 3 (sandbox host-only)**: Minimal storage: 20–25 GB. Logs kept locally and rotated frequently; no central DB usually required.

B

Appendix B: Suricata and EVE JSON (examples and recommendations)

B.1 Minimal `suricata.yaml` excerpt (EVE JSON)

```
outputs:
  - eve-log:
      enabled: yes
      filetype: regular
      filename: /var/log/suricata/eve.json
      community-id: yes
      community-id-seed: 0
      types:
        - alert:
            payload: no # set to yes only when payloads are needed
            payload-printable: no
            packet: no
            http: yes
            tls: yes
            dns: yes
        - flow:
        - http:
        - tls:
```

Notes:

- Keep `payload` disabled in normal runs to avoid large files; enable only for forensic captures.
- Use `community-id` to correlate flows across sensors.

B.2 Running Suricata (common commands)

```
# start suricata as a service (IDS mode)
sudo systemctl start suricata
sudo systemctl enable suricata
sudo systemctl status suricata

# run manually on interface (IDS passive)
sudo suricata -c /etc/suricata/suricata.yaml -i eth1

# run suricata in nfqueue/IPS mode (suricata.yaml must be configured for nfqueue)
sudo systemctl restart suricata
# ensure NFQUEUE iptables rule is present
sudo iptables -L FORWARD -n --line-numbers
```

C

Appendix C: Suricata-blocker (complete example)

A robust minimal blocker reads `/var/log/suricata/eve.json`, applies blocking rules, logs actions and performs expiry of temporary blocks. Place script at `/opt/suricata-blocker/blocker.py`.

```
#!/usr/bin/env python3
"""
Minimal Suricata-blocker
- tails /var/log/suricata/eve.json
- blocks source IPs using iptables (temporary)
- writes actions to /var/log/defender/incident.log
"""

import os, json, time, subprocess, signal, sys
from collections import defaultdict

EVE_LOG = "/var/log/suricata/eve.json"
INCIDENT_LOG = "/var/log/defender/incident.log"
BLOCK_DURATION = 10 * 60 # seconds (10 minutes)
BLOCKED = {} # src_ip -> unblock_time
RULE_WHITELIST = set() # rule ids to ignore or not block

def log_incident(msg):
    ts = time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime())
    with open(INCIDENT_LOG, "a") as f:
        f.write(f"{ts} {msg}\n")

def iptables_block(ip):
    cmd = ["sudo", "iptables", "-I", "INPUT", "-s", ip, "-j", "DROP"]
    subprocess.run(cmd)

def iptables_unblock(ip):
    cmd = ["sudo", "iptables", "-D", "INPUT", "-s", ip, "-j", "DROP"]
```

```
    subprocess.run(cmd)

def should_block(evt):
    try:
        sigid = str(evt['alert']['signature_id'])
        if sigid in RULE_WHITELIST:
            return False
        # a simple policy: block on high priority or specific sids
        sev = evt['alert'].get('severity', 0)
        return int(sev) >= 2 or sigid in ('100001','100002')
    except KeyError:
        return False

def tail(file_path):
    with open(file_path, 'r') as f:
        f.seek(0, os.SEEK_END)
        while True:
            line = f.readline()
            if not line:
                time.sleep(0.2)
                continue
            yield line

def reap_blocks():
    now = time.time()
    for ip, untime in list(BLOCKED.items()):
        if now >= untime:
            iptables_unblock(ip)
            log_incident(f"UNBLOCKED {ip}")
            del BLOCKED[ip]

def sigterm_handler(signum, frame):
    sys.exit(0)

def main():
    os.makedirs(os.path.dirname(INCIDENT_LOG), exist_ok=True)
    signal.signal(signal.SIGTERM, sigterm_handler)
    for line in tail(EVE_LOG):
        try:
            evt = json.loads(line)
        except json.JSONDecodeError:
```

```
            continue
        if evt.get('event_type') != 'alert':
            continue
        if not should_block(evt):
            continue
        src = evt.get('src_ip')
        if not src or src in BLOCKED:
            continue
        iptables_block(src)
        BLOCKED[src] = time.time() + BLOCK_DURATION
        sid = evt['alert'].get('signature_id')
        log_incident(f"BLOCKED {src} rule:{sid}")
        reap_blocks()

if __name__ == "__main__":
    main()
```

Daemonize and run:

```
sudo chmod +x /opt/suricata-blocker/blocker.py
# start in background using nohup or systemd unit
sudo nohup python3 /opt/suricata-blocker/blocker.py >/var/log/defender/blocker.ou
```

D

Appendix D: DVWA and Vulnerable Web App Deployment (examples)

D.1 DVWA quick start (Docker Compose)

A compact docker-compose file to run DVWA (recommended for reproducible labs).

```
version: '3.7'
services:
  dvwa-mariadb:
    image: mariadb:10.6
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: dvwa
      MYSQL_USER: dvwa
      MYSQL_PASSWORD: dvwa
    volumes:
      - dvwa_db:/var/lib/mysql

  dvwa:
    image: vulnerables/web-dvwa
    depends_on:
      - dvwa-mariadb
    ports:
      - "80:80"
    environment:
      - MYSQL_HOST=dvwa-mariadb
      - MYSQL_USER=dvwa
      - MYSQL_PASSWORD=dvwa
volumes:
  dvwa_db:
```

Commands:

```
cd /opt/dvwa
sudo docker-compose up -d
# Access DVWA at http://<victim-ip>/
```

D.2 Simple Flask cipher app (example)

Place this in `/opt/vulnapp/app.py` — a minimal Flask app supporting Caesar/XOR/AES-GCM endpoints (for lab use only).

```
from flask import Flask, request, jsonify
from Crypto.Cipher import AES
import base64, os

app = Flask(__name__)

def caesar_encrypt(s, shift):
    out = []
    for ch in s:
        if 'a' <= ch <= 'z':
            out.append(chr((ord(ch)-97+shift)%26 + 97))
        elif 'A' <= ch <= 'Z':
            out.append(chr((ord(ch)-65+shift)%26 + 65))
        else:
            out.append(ch)
    return ''.join(out)

@app.route('/encrypt', methods=['POST'])
def encrypt():
    cipher = request.form.get('cipher')
    text = request.form.get('text', '')
    if cipher == 'caesar':
        shift = int(request.form.get('shift', '3'))
        return jsonify({'ciphertext': caesar_encrypt(text, shift)})
    elif cipher == 'xor8':
        key = int(request.form.get('key', '0')) & 0xff
        b = bytes([c ^ key for c in text.encode()])
        return jsonify({'ciphertext': b.hex()})
    elif cipher == 'aes-gcm':
        # demo AES-GCM: use a key from env or generate
        from Crypto.Random import get_random_bytes
        key = os.environ.get('DEMO_AES_KEY')
        if not key:
```

```
            key = get_random_bytes(32)
            key_b = key
        else:
            key_b = base64.b64decode(key)
        nonce = get_random_bytes(12)
        cipherobj = AES.new(key_b, AES.MODE_GCM, nonce=nonce)
        ct, tag = cipherobj.encrypt_and_digest(text.encode())
        return jsonify({'ciphertext': (nonce+ct+tag).hex()})
    return jsonify({'error': 'unsupported cipher'}), 400

if __name__ == "__main__":
    app.run(host='0.0.0.0', port=8080)
```

D.3 Initialize DB and DVWA setup notes

```
# For system-installed DVWA (non-docker)
sudo apt update && sudo apt install apache2 mysql-server php php-mysqli
# secure mysql and create dvwa DB/user accordingly
# For Docker approach, use docker-compose above
```

E

Appendix E: Forensics, pcap capture and snapshot recovery

E.1 Packet capture for forensic replay

```
# capture traffic to pcap (run on defender monitoring interface)
sudo tcpdump -i eth1 -w /var/log/defender/test_capture.pcap

# follow file size and rotate if large
sudo timeout 3600 tcpdump -i eth1 -w /var/log/defender/test_capture_1h.pcap
```

E.2 Replaying pcap (offline analysis)

```
# replay pcap to test IDS rules (tcpreplay)
sudo tcpreplay -i eth1 /var/log/defender/test_capture.pcap
# analyze using tshark
tshark -r /var/log/defender/test_capture.pcap -T fields -e frame.time -e ip.src -
```

E.3 VM snapshot workflow (recommended)

```
# VirtualBox GUI: Right-click VM -> Snapshots -> Take snapshot (name: pre_exercis
# Revert after exercise via GUI or VBoxManage:
VBoxManage snapshot "VictimVM" restore "pre_exercise"
```

F

Appendix F: Operational scripts and utilities

F.1 Simple log rotation for EVE JSON (logrotate snippet)

```
/var/log/suricata/eve.json {
    daily
    rotate 7
    compress
    notifempty
    copytruncate
    missingok
}
```

F.2 Simple backup (tar) cron example

```
# /etc/cron.daily/backup_lab
#!/bin/bash
tar czf /var/backups/lab-$(date +%F).tgz /var/log/defender /opt/vulnapp /opt/suri
# keep retention policy in backup folder
```

G

Appendix G: Recommended software versions and packages

- Ubuntu 22.04 LTS (VMs)
- Suricata 6.x or later
- Python 3.8+
- Elasticsearch 8.x if ELK used (Type 1)
- Docker 20.10+, docker-compose 1.29+ for DVWA
- libpcap/tcpdump, tcpreplay, tshark for forensics
- PostgreSQL 12+ or newer for alert storage (optional)

H

Appendix H: Security and operational best practices (expanded)

- Run the lab in an isolated network segment (host-only or VLAN) — never expose DVWA or experiment services to production or public networks.
- Whitelist management IPs in blocker logic to prevent self-lockout of admin connections.
- Use VM snapshots or container images to quickly reset the environment.
- Limit payload logging for normal runs; enable payload capture only for deep forensics.
- Protect sensitive keys used in demos — do not store real secrets in the lab environment.
- Maintain NTP time sync across VMs (important for correlation of logs/pcaps).

Software Requirements Specification

for

<Red-Blue Lab: DVWA + Suricata + Dashboards>

Version 1.0

Prepared by:
Aryan Ranpura, Edwin Ben Paul, Aditya Bhansali
230953032, 230953021, 230953272

Organization: Manipal Institute of Technology

Date Created: November 6, 2025

Contents

1. Revision History

Version	Date	Author	Summary of changes
1.0	November 6, 2025	Aryan, Edwin, Aditya	Initial SRS prepared for submission and viva.

Contents

2. Introduction

2.1. Purpose

This SRS documents the functional and non-functional requirements for a reproducible red-team / blue-team educational lab that demonstrates web application vulnerabilities (DVWA) and defensive controls using Suricata IDS, iptables/UFW, and Flask dashboards. It serves as the authoritative specification for implementation, testing, and evaluation.

3. Introduction

3.1. Purpose

This SRS documents the functional and non-functional requirements for a reproducible red-team / blue-team educational lab that demonstrates web application vulnerabilities (DVWA) and defensive controls using Suricata IDS, iptables/UFW and simple Flask dashboards. It serves as the authoritative specification for implementation, testing and evaluation.

3.2. Document conventions

- Requirement identifiers are of the form `REQ-<section>-<number>`, e.g. `REQ-4.3-01`.
- Priority levels: **High**, **Medium**, **Low**.
- Paths and examples assume Linux guest VMs (Ubuntu Kali). Adjust usernames/paths if different.

3.3. Intended audience and reading suggestions

Intended readers: developers, evaluators/instructors, testers. Developers should focus on Sections 3–5; evaluators on the introduction, product scope, STRIDE/risk, and demo checklist; testers on functional requirements and acceptance criteria.

3.4. Product scope

A self-contained lab comprised of two VirtualBox VMs (Attacker: Kali; Defender: Ubuntu) on an Internal Network. The Defender hosts DVWA, Suricata, a Secret Manager for encrypted demo data, and a Defence Dashboard. The Attacker runs offensive tools and an Attacker Dashboard. The lab is isolated from host network by VirtualBox Internal Network and disabled guest integrations (no shared folders/clipboard).

3.5. References

- DVWA (Damn Vulnerable Web Application) documentation.
- Suricata documentation and `/etc/suricata/suricata.yaml`.

- Project code files under `/home/aryan/defence/` and `/attacker`$_d$$ashboard/$.

4. Overall description

4.1. Product perspective

This lab is modular: an Attacker VM (Kali) and Defender VM (Ubuntu). Network traffic between VMs is inspected by Suricata on Defender. Alerts are stored in JSON and consumed by a watcher and dashboard. A Secret Manager produces encrypted secret files that the attacker can fetch from Apache.

4.2. Product functions (high level)

- Provide a vulnerable web target (DVWA) on Defender for attack demonstrations.
- Capture and detect attacks using Suricata with local rules.
- Persist alerts to `eve.json` and present them via the Defence Dashboard.
- Optionally auto-block malicious source IPs with iptables; provide safe toggle/cooldown behavior.
- Provide an Attacker Dashboard for logging and fetching encrypted secret files.
- Secret Manager to generate encrypted files using simple classical ciphers (Caesar, Vigenère, Autokey, Railfence, Affine, Atbash, ROT13).

4.3. User classes and characteristics

- Evaluator / Instructor: inspects, runs demos, grades.
- Student / Developer: implements and modifies code, writes rules.
- Attacker (demo role): runs attacks and demonstrates exfiltration.
- Automated scripts: demo orchestration, tests.

4.4. Operating environment

Host: Windows laptop with VirtualBox; Guest OS: Ubuntu (Defender), Kali (Attacker). Software: Apache2, PHP, MySQL, Suricata, Python3, Flask, iptables/nft.

4.5. Design and implementation constraints

Offline operation by default; temporary NAT allowed for package installs. Suricata configured to use the VM internal NIC with `af-packet`. Auto-block requires root.

4.6. User documentation

README, demo scripts, cheat-sheet for viva, and brief user guides for starting services and restoring snapshots.

4.7. Assumptions and dependencies

VirtualBox available, system resources sufficient (16GB recommended), Suricata and Python packages installed.

5. External interface requirements

5.1. User interfaces

Web UIs (Flask):

- Defence Dashboard: alerts table, blocked IPs, charts, block/unblock controls.
- Secret Manager: create secrets, list files, decrypt preview (host-only).
- Attacker Dashboard: log attacks, fetch secret files (attacker VM only).

5.2. Hardware interfaces

Virtual NICs provided by VirtualBox.

5.3. Software interfaces

Suricata writes alerts to `/var/log/suricata/eve.json`. Dashboards read and write JSON files in the defence directory. Apache serves static secret files from `/var/www/html`.

5.4. Communications interfaces

HTTP for DVWA and secret files, Flask dashboards on configurable local ports. All traffic within VirtualBox Internal Network.

6. System features

6.1. DVWA Web Application

Priority: High.

- **REQ-4.1-01 (High)**: DVWA must be reachable at `http://10.10.10.11/DVWA`.
- **REQ-4.1-02 (Medium)**: DVWA should allow changing security level (Low/Medium/High).
- **REQ-4.1-03 (Low)**: DVWA logs must be available for forensic review.

6.2. Suricata IDS & Rules Engine

Priority: High.

- **REQ-4.2-01 (High)**: Suricata must run in `af-packet` mode bound to the internal NIC.
- **REQ-4.2-02 (High)**: Alerts in `eve.json` must include timestamp, src/dst IP, alert.signature, event_type.
- **REQ-4.2-03 (Medium)**: Local rule file `/etc/suricata/rules/local.rules` must be editable and reloadable.
- **REQ-4.2-04 (Medium)**: Suricata must pass `suricata -T` and restart correctly.

6.3. Defence Dashboard (Flask)

Priority: High.

- **REQ-4.3-01 (High)**: Display recent incidents (e.g., last 200) from `incidents.jsonl`.

- **REQ-4.3-02 (High)**: Display the list of blocked IPs read from `blocked`$_i ps.json$.
- `REQ-4.3-03 (Medium): Provide an Unblock endpoint that removes all DROP rules for an IP and records recently-unblocked timestamp.`
- `REQ-4.3-04 (Medium): Provide a UI toggle to simulate block vs perform real iptables actions.`
- `REQ-4.3-05 (Low): Allow export/download of incident logs.`

6.4. Incidents Watcher / Auto-blocker

Priority: High (auto-block behavior), Medium (watcher).

- **REQ-4.4-01 (High)**: Tail `eve.json` reliably and append incidents to `incidents.jsonl`.
- **REQ-4.4-02 (High)**: Insert iptables drop rule `-I INPUT -s <src`$_i p > -jDROPwhenauto-blockenabled$.
- `REQ-4.4-03 (High): Skip blocking IPs recently unblocked within a configurable COOLDOWN (default 60s).`
- `REQ-4.4-04 (Medium): Persist blocked`$_i ps.jsonatomically$.

6.5. Attacker Dashboard (Kali)

Priority: Medium.

- **REQ-4.5-01 (High)**: Allow fetching a URL (http) and display response body in the UI.
- **REQ-4.5-02 (Medium)**: Allow logging manual attack entries to `attacks.jsonl`.
- **REQ-4.5-03 (Low)**: Provide a copy/paste-friendly view of fetched ciphertext.

6.6. Secret Manager (Cipher generator)

Priority: Medium.

- **REQ-4.6-01 (High)**: Support Caesar, Autokey, Vigenère, Railfence, Affine, Atbash, ROT13 encryption.
- **REQ-4.6-02 (Medium)**: Save files to `/var/www/html` with a predictable naming pattern: `secret_<timestamp>_<cipher>.enc`.
- **REQ-4.6-03 (Low)**: Provide decrypt preview (host-only UI).
- **REQ-4.6-04 (Low)**: Validate parameters (e.g. Affine 'a' coprime with 26).

7. Other non-functional requirements

7.1. Performance requirements

- **NFR-5.1-01 (Medium)**: Suricata should process lab-level traffic (up to several hundred pps) without significant packet drops on the VM given adequate resources.
- **NFR-5.1-02 (Low)**: Dashboards should render within 2 seconds on the VM.

7.2. Safety requirements

- **NFR-5.2-01 (High)**: The lab must be isolated (VirtualBox Internal Network) and guest integrations disabled.
- **NFR-5.2-02 (High)**: Snapshots must be taken before demos to allow safe rollback.

7.3. Security requirements

- **NFR-5.3-01 (High)**: Auto-block must be disableable. Default demo mode recommended: auto-block disabled.
- **NFR-5.3-02 (Medium)**: Secret Manager UI binds to localhost by default; if exposed to internal network, add minimal auth.
- **NFR-5.3-03 (High)**: All critical iptables actions must be auditable (logs).

7.4. Software quality attributes

- Maintainability: modular code for ciphers, watcher, UI.
- Usability: color-coded dashboards with clear actions.
- Portability: runs on Ubuntu and Kali with Python3.

7.5. Business rules

- The lab is for educational/demonstration use only; no unauthorized attacks.
- Evaluator must ensure snapshots are taken before demos.
- Auto-block should not be left enabled unsupervised.

8. Appendix A: Glossary

DVWA: Damn Vulnerable Web Application.

Suricata: Network intrusion detection and prevention system.

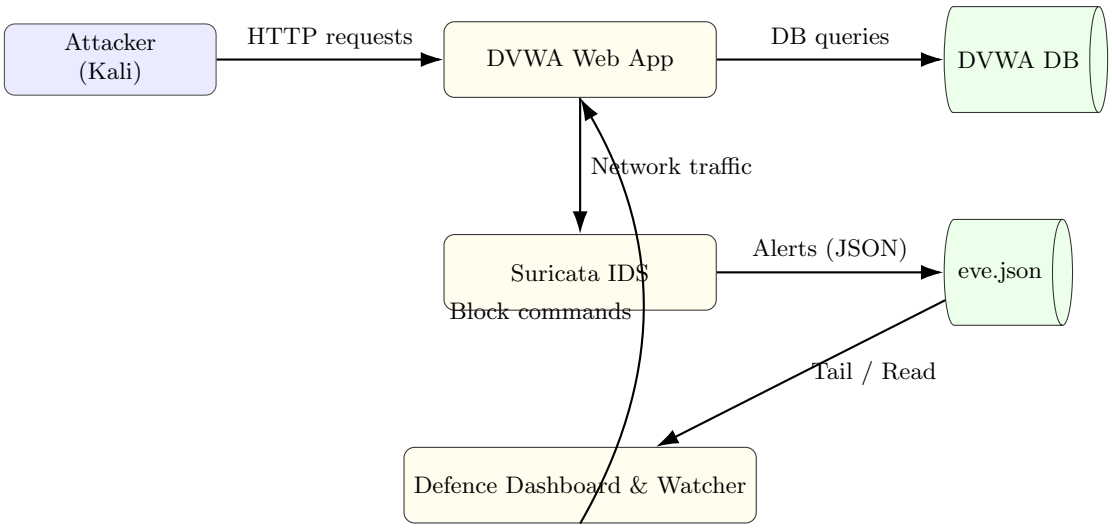
eve.json: Suricata JSON event log.

Watcher: incidents_writer_auto.py that tails eve.json.

Auto-block: automatic insertion of iptables DROP rules based on alerts.

9. Appendix B: Analysis models

Below is a small Data Flow Diagram (DFD) illustrating the main components and data flows. (You can replace or expand this diagram with the TikZ DFD in the threat-model document.)



10. Appendix C: To Be Determined list

- Whether Secret Manager should write files as `www-data` or write to home and copy with sudo (TDB-1).
- Add optional authentication to dashboards for classroom use (TDB-2).
- Implement evidence export (pcap + eve slice) script (TDB-3).

11. Demo checklist (short)

1. Start Defender (Ubuntu) and Attacker (Kali) VMs on Internal Network with IPs (e.g., 10.10.10.11 and 10.10.10.10).
2. Start Apache + DVWA on Defender.
3. Start Suricata: `sudo systemctl start suricata`.
4. Start defence watcher and dashboard (or systemctl services if configured).
5. On Kali, perform a simple SQLi: `curl "http://10.10.10.11/DVWA/vulnerabilities/sqli/?id=1' OR '1'='1Submit=Submit"`.
6. Observe Suricata alert in `/var/log/suricata/eve.json` and Defence Dashboard.
7. If manual blocking: `sudo iptables -I INPUT -s 10.10.10.10 -j DROP` and show attacker fails.
8. Restore snapshot after demo.

12. Acceptance criteria

- DVWA reachable and exploitable per demo scenarios.
- Suricata logs alerts for test signatures (local.rules) demonstrably appear in `eve.json`.
- Defence Dashboard shows alerts and blocked IPs; Unblock API reliably removes iptables DROP rules.
- Secret Manager can produce files accessible from attacker via HTTP and Defender can decrypt them via UI with correct params.

13. Contact

Authors:
Aryan Ranpura, Edwin Ben Paul, Aditya Bhansali
230953032, 230953021, 230953272